

SQL vs NoSQL in System Design

 systemdesignschool.io/fundamentals/sql-vs-nosql



SQL vs NoSQL

We've explored SQL and NoSQL databases separately. SQL provides ACID transactions and relational queries. NoSQL offers flexible schemas and horizontal scaling. How do you choose between them?

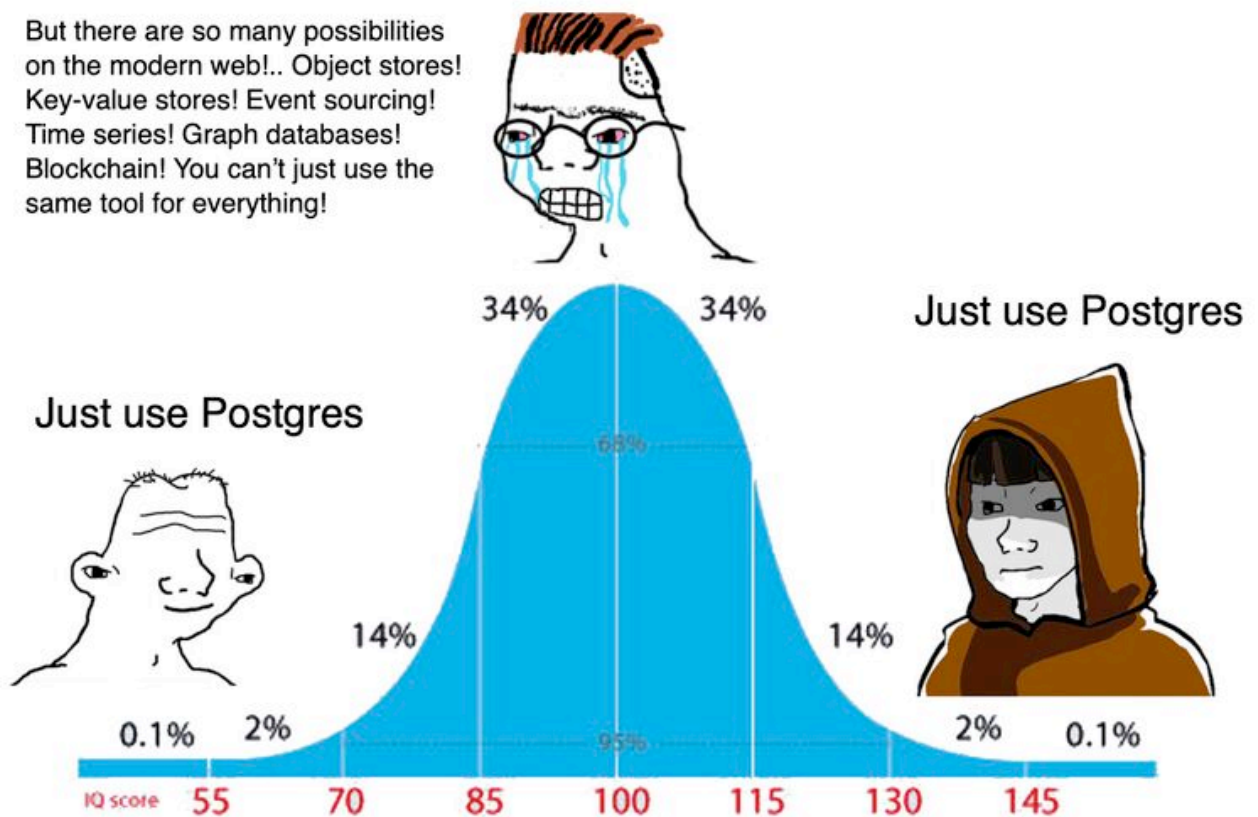
How to Choose Between SQL and NoSQL

Most of the Time, Either Works

Many systems can be built using either SQL or NoSQL, and the choice may come down to developer familiarity or infrastructure preferences.

There is a popular meme ["Just use PostgreSQL"](#) that is a reference to this statement.

But there are so many possibilities on the modern web!.. Object stores! Key-value stores! Event sourcing! Time series! Graph databases! Blockchain! You can't just use the same tool for everything!



The line is blurry, and traditional RDBMS like PostgreSQL also support some NoSQL features such as [JSON fields](#) that you can even [query](#) on.

This is especially true for small systems where it's not necessary to toil over the choices. If the QPS is low, it's important to choose the simpler solution and not over-engineer.

If You Need Transactions, Go with SQL

SQL databases are better suited for use cases requiring strong consistency and ACID transactions.

Example: Banking systems and e-commerce checkout flows need strict transactional integrity.

Data Model

- Is the data structured and relational? If yes, SQL is preferred.
- If the data is semi-structured or unstructured, NoSQL is more suitable.

Consistency vs Availability

- Does the system require strong consistency (e.g., financial transactions)? Use SQL.

- If the system prioritizes availability (e.g., content feeds), NoSQL is the better option.

Query Requirements

- If the system involves complex queries or relational operations (e.g., JOINS), SQL is ideal.
- If the system focuses on fast lookups or simple key-value operations, NoSQL is a better fit.

Hybrid Systems: Multiple Databases in a Single System

In modern systems, it's common to use multiple databases (even of different types) for specific components.

- Example: A large e-commerce website might use:
 - SQL (PostgreSQL) for managing orders and transactions
 - NoSQL (Redis) for session storage and caching
 - Document Store (MongoDB) for product catalog management
- Example: A social network might use:
 - SQL (PostgreSQL) for user information
 - NoSQL (DynamoDB) for friendship relations
 - NoSQL (Redis) for storing pending user feeds